

DW1000 Driver Library v1.3.01 – Описание и руководство пользователя

Оглавление

1. Назначение библиотеки	2
2. Состав библиотеки	3
3. Основные модули и их функции	3
3.1 Драйвер DW1000 (`dw1000_driver.c/.h`)	3
3.2 Протокол связи с MATLAB (`protocol.c/.h`)	4
3.3 Менеджер радио (`radio_manager.c/.h`)	4
3.4 Отладочная консоль (`debug_console.c/.h`)	4
3.5 USB CDC (`usbd_cdc_if.c/.h`)	5
4. Рекомендации по интеграции	5
4.1 Подключение библиотеки в проекте STM32CubeIDE	5
4.2 Порядок инициализации	5
4.3 Работа с двумя модулями DW1000 для МКС	6
4.4 Обработка прерываний	6
4.5 Отладочная консоль	7
5. Зависимости	7
6. Версионность	7
7. Ограничения и планы развития	8
8. Пример использования отладочной консоли	8

1. Назначение библиотеки

Библиотека предоставляет программный интерфейс для управления модулями DecaWave DW1000 в проекте МКС (Модуль коммуникации и сопряжения).

Библиотека DW1000 Driver Library v1.3.01 реализует:

- Полный цикл инициализации и конфигурации чипа DW1000.
- Низкоуровневый доступ к регистрам через SPI с поддержкой сложных заголовков (1-3 байта).
- Управление передачей и приёмом кадров, включая асинхронные операции с использованием FreeRTOS.
- Сбор диагностической информации (CIR, метрики сигнала) и её передачу по запросу.
- Обработку прерываний от DW1000 и синхронизацию с задачами FreeRTOS через семафоры и очереди.
- Реализацию бинарного протокола связи с MATLAB (версия 1.3).
- Отладочную консоль через UART для мониторинга и управления системой в реальном времени.

Версия библиотеки 1.3.01 синхронизирована с версиями документов: «МКС API v.1.3.pdf», «Бинарный протокол МКС v.1.3.pdf» и «DW1000 User Manual v.2.17».

2. Состав библиотеки

Файл	Назначение
dw1000_driver.h/.c	Драйвер низкого уровня: работа с регистрами, инициализация, загрузка LDE, сброс, обработка прерываний
protocol.h/.c	Реализация бинарного протокола связи с MATLAB: парсер пакетов, диспетчер команд, построитель ответов, обработчики всех команд API
radio_defs.h	Определения структур для команд и событий, используемых в очередях FreeRTOS
radio_manager.h/.c	Задача управления радио (менеджер), обрабатывающая команды из очереди `xRadioCommandQueue` и события из `xRadioEventQueue`
debug_console.h/.c	Отладочная консоль через UART: инициализация, задача консоли, функции вывода, парсер команд оператора
main.c	Пример главного файла с инициализацией системы, созданием очередей и задач
main.h	Заголовочный файл с глобальными определениями и extern-переменными
usbd_cdc_if.c/.h	Реализация интерфейса USB CDC (Virtual COM Port) для связи с MATLAB

3. Основные модули и их функции

3.1 Драйвер DW1000 (`dw1000\_driver.c/.h`)

- Инициализация устройства – `DW1000\_Init()`: аппаратный сброс, проверка идентификатора, загрузка LDE-микрокода, настройка по умолчанию.
- Доступ к регистрам – набор функций `DW1000\_ReadRegister`, `DW1000\_WriteRegister` и их обёртки для 8/16/32-битных значений.
- Сброс – `DW1000\_SoftReset()` (receiver-only reset).
- Управление передачей – `DW1000\_SendFrame()` (запись данных в буфер, установка длины, запуск).
- Установка мощности – `DW1000\_SetTxPower()`.
- Чтение OTP – `DW1000\_ReadOTP()` для калибровочных значений.
- Обработчик прерываний – `DW1000\_IRQHandler()`, вызываемый из внешнего прерывания, анализирует `SYS\_STATUS`, отправляет события в очередь

`xRadioEventQueue` и даёт семафоры `xTxCompleteSemaphore` / `xRxCompleteSemaphore`.

3.2 Протокол связи с MATLAB (`protocol.c/.h`)

- Парсер – принимает байты из USB, ищет синхрослово `0xAA55`, проверяет CRC8, извлекает команду и параметры.
- Диспетчер – на основе `CMD\_ID` вызывает зарегистрированный обработчик.
- Обработчики команд – реализуют все 16 команд API (PING, INIT, GET_STATUS, SET_PHY_CONFIG, TX_FRAME, RX_START, GET_SIGNAL_METRICS, GET_CIR и др.). Команды, требующие длительных операций, отправляют задание в очередь `xRadioCommandQueue` и ожидают семафор завершения.
- Построитель ответов – формирует ответный пакет с кодом статуса и данными, помещает его в очередь `xUSB\_TxQueue` для отправки через USB.

3.3 Менеджер радио (`radio\_manager.c/.h`)

- Задача `RADIO\_Manager\_Task` – запускается как отдельная задача FreeRTOS. Ожидает команды из глобальной очереди `xRadioCommandQueue`, вызывает соответствующие обработчики (`handle\_tx\_frame`, `handle\_rx\_start` и др.), которые взаимодействуют с драйвером DW1000.
- Обработка событий – также получает события из очереди `xRadioEventQueue` (сгенерированные в прерываниях) и выводит отладочную информацию через консоль.
- Инкапсуляция – все операции с DW1000, требующие синхронизации, выполняются в контексте этой задачи, что предотвращает конфликты доступа.

3.4 Отладочная консоль (`debug\_console.c/.h`)

- Инициализация – `DEBUG\_CONSOLE\_Init(UART\_HandleTypeDef\* huart)` связывает консоль с выбранным UART.
- Задача консоли – `DEBUG\_Console\_Task` читает символы из UART, накапливает строку, по нажатию Enter выполняет разбор и вызов соответствующей команды оператора.
- Функции вывода – `DEBUG\_Print`, `DEBUG\_Println`, `DEBUG\_Printf`, `DEBUG\_PrintHex`, `DEBUG\_PrintStatus` – для отладочной печати из любого места кода.

- Уровни отладки – ``DEBUG_SetLevel()`` позволяет фильтровать сообщения по важности.

- Команды оператора – набор встроенных команд (help, status, tx, rx, power, channel, reset, log, cir, metrics, exp) для управления системой без MATLAB.

3.5 USB CDC (``usbd_cdc_if.c/h``)

- Реализует кольцевой буфер для приёма данных от MATLAB.

- Предоставляет функции ``CDC_ReceiveByte`` (неблокирующее чтение) и ``CDC_Transmit`` (отправка данных).

4. Интеграция

4.1 Подключение библиотеки в проекте STM32CubeIDE

1. Скопировать папку с библиотекой в ``Middlewares/DW1000_Lib``.
2. В настройках проекта добавить путь к заголовочным файлам: ``Middlewares/DW1000_Lib/inc``.
3. Добавить все ``.c`` файлы из ``src`` в проект (например, в отдельную группу "DW1000_Lib").
4. Убедиться, что в проекте включена поддержка FreeRTOS и правильно настроены очереди/семафоры.
5. В ``main.c`` выполнить инициализацию периферии (CubeMX), затем вызвать функции инициализации библиотеки и создать задачи согласно примеру.

4.2 Порядок инициализации

// После инициализации HAL и периферии:

```
DEBUG_CONSOLE_Init(&huart2);      // инициализация отладочной консоли
```

```
DW1000_ConfigureDevice1(&tx_device);
```

```
DW1000_ConfigureDevice2(&rx_device);
```

```
spi_set_rate_low();               // сначала делаем медленный SPI для инициализации
```

```
DW1000_Init(&tx_device);
```

```
DW1000_Init(&rx_device);
```

```
spi_set_rate_high();           // быстрый SPI после инициализации
PROTOCOL_Init();
PROTOCOL_RegisterAllHandlers();
```

```
// Создание очередей и семафоров
```

```
xRadioCommandQueue = xQueueCreate(10, sizeof(RadioCommand_t*));
xRadioEventQueue = xQueueCreate(10, sizeof(RadioEvent_t));
xUSB_TxQueue = xQueueCreate(10, sizeof(TxBuffer_t*));
xConsoleCommandQueue = xQueueCreate(10, sizeof(char[64]));
xTxCompleteSemaphore = xSemaphoreCreateBinary();
xRxCompleteSemaphore = xSemaphoreCreateBinary();
```

```
// Создание задач FreeRTOS
```

```
xTaskCreate(USB_Command_Task, "USB_Cmd", 256, NULL, 2, NULL);
xTaskCreate(USB_Transmit_Task, "USB_Tx", 256, NULL, 2, NULL);
xTaskCreate(RADIO_Manager_Task, "RadioMgr", 512, NULL, 3, NULL);
xTaskCreate(Periodic_TX_Task, "PerTX", 256, NULL, 1, NULL);
xTaskCreate(Diagnostic_Stream_Task, "Diag", 256, NULL, 1, NULL);
xTaskCreate(DEBUG_Console_Task, "Console", 512, NULL, 1, NULL);
```

```
vTaskStartScheduler();
```

4.3 Работа с двумя модулями DW1000 для МКС

Использовать две отдельные структуры `DW1000\_Device` с разными пинами CS, IRQ и RST. SPI-шина общая, все операции чтения/записи защищены критическими секциями внутри драйвера.

4.4 Обработка прерываний

В файле `stm32f4xx\_it.c` необходимо вызвать `DW1000\_IRQHandler` для соответствующего устройства:

```

void EXTI0_IRQHandler(void) {
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_0) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_0);
        DW1000_IRQHandler(&tx_device);
    }
}

void EXTI1_IRQHandler(void) {
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_1) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_1);
        DW1000_IRQHandler(&rx_device);
    }
}

```

4.5 Отладочная консоль

Подключить UART к терминалу (115200, 8N1). После запуска появится приглашение `>`. Доступные команды можно посмотреть по команде `help`. Консоль позволяет выполнять те же операции, что и MATLAB, но вручную, для отладки.

5. Зависимости

- STM32 HAL (для SPI, GPIO, UART, USB).
- FreeRTOS (для очередей, семафоров, задач).
- Стандартные библиотеки C (`string.h`, `stdlib.h`, `stdarg.h`, `stdio.h`).

6. Версионность

Версия	Дата	Изменения
1.0		Первая реализация драйвера и протокола
1.1		Добавлена поддержка двух модулей DW1000
1.2		Интеграция с FreeRTOS (очереди, семафоры)

1.3		Полная синхронизация с API v1.3, реализованы все команды
1.3.01	текущая	Добавлена отладочная консоль через UART (debug_console). Улучшена обработка ошибок с выводом в консоль.

7. Ограничения

- Не реализовано: автоматическая калибровка антенной задержки ('TX_ANTD', 'LDE_RXANTD') (АНТОНОВ)
- Не реализовано: режимы пониженного энергопотребления (SNIFF, DEEPSLEEP) (САВИН)
- Требуется доработки: полная настройка PHY в 'DW1000_SetPhyConfig' (сейчас только кэширует параметры, запись регистров частично реализована в 'radio_manager.c') (Внимание! САВИН)
- Планируется: поддержка чтения температуры и напряжения (TC_SAR), а также режима свипирования ('TX_SWEEP') (АНТОНОВ телеметрия, САВИН режимы)

8. Пример использования отладочной консоли

После подключения к UART и запуска системы можно вводить команды:

```
> status
```

```
=== Состояние МКС ===
```

```
TX: channel=5, rate=2, preamble=128, PRF=1
```

```
RX: channel=5, rate=2, preamble=128, PRF=1
```

```
Experiment running: NO
```

```
> tx 4 01020304
```

```
Передача 4 байт... ОК
```

```
> rx on
```

```
Включение приёмника...
```

```
> metrics
```

```
Метрики последнего пакета:
```

RXPACC: 128

FP_INDEX: 45

FP_AMPL1: 32000

STD_NOISE: 120

> exp start

Эксперимент запущен